

MODELOS DE PROGRAMACIÓN ORIENTADA A OBJETOS · GRUPO 6

Tema 4 · Estructuras de selección

Que el programa decida: `if`, ternario y `switch`

LA GRAN IDEA

Del flujo recto... a las decisiones

Hasta ahora el programa corría **línea por línea**. Hoy aprende a **bifurcarse**: tomar un camino u otro según una condición.

A · Booleanos e if 30'

B · Ternario 10'

C · switch y enum 20'

D · Anidar y Math 10'

E · Código en vivo y cierre 30'

DE LA CLASE PASADA

Ya usaron una condición sin decirlo

El lunes descubrimos que `0.1 + 0.2 == 0.3` es `false`, y que dos cadenas iguales pueden no ser el mismo objeto. Las dos son **preguntas que devuelven true o false**. Hoy esas preguntas deciden por dónde sigue el programa.

```
System.out.println(0.1 + 0.2 == 0.3);    // false
System.out.println(a == b);              // compara objetos, no texto
System.out.println(a.equals(b));         // compara el contenido
```

SECCIÓN A

Decisiones con `if`

🕒 ~30 min

EL MOTOR DE LA DECISIÓN

La condición es un booleano

Toda selección se apoya en una **expresión booleana** (de los operadores que ya vimos): produce `true` o `false`, y eso decide el camino.

```
boolean mayorDeEdad = (edad >= 18);  
boolean enRango = (n > 0) && (n < 100);
```

SELECCIÓN · CÓMO DECIDE EL IF

El booleano decide el camino

condición: `edad >= 18`



TRUE Mayor de edad



FALSE Menor de edad

La condición produce true o false; solo se ejecuta una rama.

```
if (edad >= 18) {  
    System.out.println("Mayor de edad");  
} else {  
    System.out.println("Menor de edad");  
}
```

LA ESTRUCTURA BÁSICA

if / if-else

```
if (edad >= 18) {  
    System.out.println("Mayor de edad");  
} else {  
    System.out.println("Menor de edad");  
}
```

Si la condición es `true`, corre el primer bloque; si no, el `else`.

La escalera `if-else-if`

```
char letra;  
if (calif >= 90) {  
    letra = 'A';  
} else if (calif >= 80) {  
    letra = 'B';  
} else if (calif >= 70) {  
    letra = 'C';  
}
```

Se evalúan en orden; la **primera** verdadera gana y las demás se ignoran.

calif >= 90 ?

true → letra = 'A' · GANA

calif >= 80 ?

ya no se evalúa

calif >= 70 ?

ya no se evalúa

else

letra = 'F'

Se evalúan en orden; la primera verdadera gana.

TRUCO PROFESIONAL

Operadores lógicos y *short-circuit*

```
// Si nombre es null, la 2a parte NO se evalúa:  
// así evitamos un NullPointerException  
if (nombre != null && nombre.length() > 0) {  
    System.out.println(nombre);  
}
```

&& se detiene si la izquierda es false; || si es true. Se usa a diario para proteger el código.

nombre != null
se evalúa → false

&&

nombre.length() > 0
✗ no se evalúa

&& se detiene si la izquierda es false; así evitas el NullPointerException.

BUENAS PRÁCTICAS · HISTORIA REAL

Usa llaves { } SIEMPRE

```
if (claveValida)
    acceso = true;
    registrarEntrada();    // BUG: esto se ejecuta SIEMPRE (no esta en el if)
```

En 2014, un `if` sin llaves con una línea duplicada (`goto fail;`) rompió la validación de seguridad **SSL en millones de dispositivos Apple**. La regla de oro: **llaves siempre, aunque sea una sola línea**.

LAS DOS TRAMPAS CLÁSICAS

= vs == • y comparar Strings

```
if (x == 5) { }           // COMPARA (correcto)
boolean b = false;
if (b = true) { }         // OJO: ASIGNA, no compara (bug con boolean)

String a = new String("hola");
System.out.println(a == "hola");           // false (compara referencias!)
System.out.println(a.equals("hola"));       // true  (compara contenido)
```

En Java, los Strings **siempre** se comparan con `.equals()`.

 Tu turno · 5 min

¿Año bisiesto?

Escribe una condición que diga si un año es bisiesto:

Es bisiesto si es **divisible entre 4, excepto** los divisibles entre 100... **salvo** que también lo sean entre 400.

Pista: combina %, ==, !=, && y ||.

SELECCIÓN COMPACTA

El operador ternario ? :

```
// forma larga
String tipo;
if (edad >= 18) { tipo = "adulto"; } else { tipo = "menor"; }

// forma corta (ternario): condicion ? valorSi : valorNo
String tipo2 = (edad >= 18) ? "adulto" : "menor";
int max = (a > b) ? a : b;
```

Ideal para **asignar un valor** según una condición. No abuses: si crece, mejor un `if`.

SECCIÓN C

La estructura switch

🕒 ~20 min

COMPARAR UNA VARIABLE CONTRA VALORES

switch clásico

```
switch (opcion) {  
    case 1:  
        System.out.println("Depositar");  
        break;  
    case 2:  
        System.out.println("Retirar");  
        break;  
    default:  
        System.out.println("Opcion invalida");  
}
```

break corta el switch; default es el "ninguno de los anteriores".

EL ERROR QUE TODOS COMETEN

La trampa del *fall-through*

```
switch (dia) {  
    case 1:  
        System.out.println("Lunes");  
        // sin break: el flujo SIGUE al siguiente case!  
    case 2:  
        System.out.println("Martes");  
        break;  
}  
// Con dia = 1 imprime: Lunes Y Martes
```

Olvidar un break es una fuente clásica de bugs.

JAVA ACTUAL (21) · LO QUE SE USA HOY

switch moderno con flecha

```
String nombre = switch (dia) {  
    case 1 -> "Lunes";  
    case 2 -> "Martes";  
    case 6, 7 -> "Fin de semana";  
    default -> "Entre semana";  
};
```

Sin break (no hay fall-through) · es una **expresión** que devuelve un valor · el compilador exige cubrir todos los casos. **Más seguro y más limpio.**

TAMBIÉN CON TEXTO

switch sobre String

```
switch (comando) {  
    case "start" -> iniciar();  
    case "stop"  -> detener();  
    default      -> System.out.println("Comando desconocido");  
}
```

En Java 21 el switch hasta reconoce **tipos** (*pattern matching*), que verán en materias avanzadas.



El switch enruta el valor al case que coincide y salta directo a él; el break termina el switch.

VALORES CON NOMBRE

enum: el conjunto cerrado de casos

Un **enum** es un tipo con una lista fija de valores. Dentro de un `switch` se leen solos, y nadie puede pasar un valor que no exista.

```
enum Dia { LUNES, MARTES, MIERCOLES, JUEVES, VIERNES }

Dia hoy = Dia.LUNES;

switch (hoy) {
    case LUNES      -> System.out.println("Teoria, en linea");
    case MIERCOLES  -> System.out.println("Laboratorio, en Q007");
    default         -> System.out.println("Ni teoria ni laboratorio");
}
```

Dentro del `switch` se escribe `LUNES`, sin repetir `Dia`. Compara: con `int` o `String` cualquier valor cabe; con `enum`, solo los cinco.

¿CUÁL USO?

if-else-if vs switch

Usa if-else-if

Para **rangos** y condiciones complejas: `calif >= 90`, combinaciones con `&&/|`.

Usa switch

Para comparar **una variable** contra **valores concretos** (menús, días, estados). Más legible con muchos casos.

SECCIÓN D

Anidar, ordenar y la clase Math

🕒 ~10 min

QUE SE LEA DE CORRIDO

Anidamiento y salidas tempranas

Un `if` dentro de otro se vuelve ilegible en cuanto pasas de dos niveles. Casi siempre queda mejor **sacar primero los casos imposibles** y dejar el caso bueno hasta abajo, sin sangría.

```
// anidado
if (cuenta != null) {
    if (saldo >= monto) {
        retirar(monto);
    }
}
```

```
// salidas tempranas
if (cuenta == null) return;
if (saldo < monto) return;

retirar(monto);
```

Misma lógica, la mitad de sangría: cada condición dice de qué te protege.

La clase Math

Métodos `static`: se llaman con el nombre de la clase, sin crear ningún objeto.

Aparecen todo el tiempo dentro de las condiciones.

```
Math.abs(-7)           // 7
Math.max(a, b)         // el mayor de los dos
Math.min(a, b)         // el menor
Math.round(3.6)        // 4
Math.pow(2, 10)         // 1024.0
Math.sqrt(144)          // 12.0
Math.random()          // un decimal entre 0.0 y 1.0

if (Math.abs(x - y) < 0.0001) { ... } // así SI se comparan decimales
```

La última línea cierra lo del lunes: los decimales no se comparan con `==`, se compara su **diferencia** contra una tolerancia.

SECCIÓN E

Manos al código y cierre

🕒 ~30 min

Menú interactivo con **switch**

```
import java.util.Scanner;

public class Menu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("1) Saludar    2) Despedir");
        int op = sc.nextInt();
        switch (op) {
            case 1 -> System.out.println("Hola!");
            case 2 -> System.out.println("Adios!");
            default -> System.out.println("Opcion invalida");
        }
        sc.close();
    }
}
```

En el IDE: lo corremos con 1, 2 y 9, y comparamos con la versión clásica (con break).

 Tu turno · 8 min

Clasificador de calificaciones

Lee una calificación (de 0 a 100) con Scanner y muestra su letra (A/B/C/D/F) usando **if-else-if**. Bonus: hazlo también con **switch moderno**.

Lo comparamos en pantalla al terminar.

Pista: para switch por rangos, divide entre 10 y evalúa el resultado.

ANTES DE QUE TE PASE

Errores comunes

- Comparar Strings con `==` en vez de `.equals()`.
- Olvidar `break` en el `switch` clásico (fall-through).
- `if` sin llaves + una línea extra → bug silencioso.
- Confundir `=` (asigna) con `==` (compara).

MINI-QUIZ DE COMPRENSIÓN

¿Nos quedó claro?

1. ¿Qué imprime un `switch` clásico si olvidas el `break`?
2. ¿Por qué `"a" == "a"` puede fallar y `.equals()` no?
3. Menciona una ventaja del `switch` moderno (flecha) sobre el clásico.

CIERRE

Lo que te llevas + tarea

- Toda selección se apoya en un **booleano**; `if/else-if` para condiciones y rangos.
- Ternario para asignar un valor; `switch` para una variable contra valores.
- Usa el **switch moderno** (flecha) y compara Strings con `.equals()`. Llaves siempre.
- `enum` cuando los casos son un conjunto cerrado; `Math` para las cuentas dentro de la condición.
- Antes de anidar, pregunta si cabe una **salida temprana**.

 **Para practicar** (no se entrega): el clasificador de calificaciones, en las dos formas. ➡ **Miércoles 23:** P4, variables y arreglos, presencial en Q007.